PLACEMENT & INTERVIEW PREP

Agentic RAG (with LangGraph)

RAG + an autonomous decision-making agent — explained simply, with code & diagrams

```
Query → Agent decides → Retrieve OR skip → Generate → Answer
```

1. What is Agentic RAG?

IN ONE LINE

Agentic RAG = **RAG** + an **autonomous agent**. Instead of always retrieving context in a fixed order, an agent gets to make intelligent, dynamic decisions about whether and how retrieval should happen.

Two components work together here:

- **RAG** — gives access to external information: a vector database, an API, a tool, or another data source
- **Autonomous agent** — decides *how* that information/retrieval process should actually be used

2. Traditional RAG vs Agentic RAG

Traditional RAG

A fixed flow that always runs the same way:

```
Query → Retrieve → Context → Prompt → LLM → Answer
```

Agentic RAG

Adds a decision step before anything else happens:

```
Query → Agent decides → Retrieve OR Generate → Answer
```

The key difference: the agent can dynamically choose which path to take, instead of always retrieving whether it's needed or not.

3. Four Decisions the Agent Can Make

Decision	Question it answers
When to retrieve	Is retrieval even necessary for this question?
What to retrieve	Which specific information is relevant?
Where to retrieve	Which source/tool/database should be queried?
How many times to retrieve	Is one retrieval round enough, or are more needed?

These four decisions are the heart of what makes a RAG system "agentic" rather than fixed.

4. Why LangGraph?

LangGraph is used to actually build this agentic workflow. Think of it as drawing a flowchart in code: **nodes** do work, and **edges** define how execution moves from one node to the next. A **conditional edge** picks the next node based on the current state.

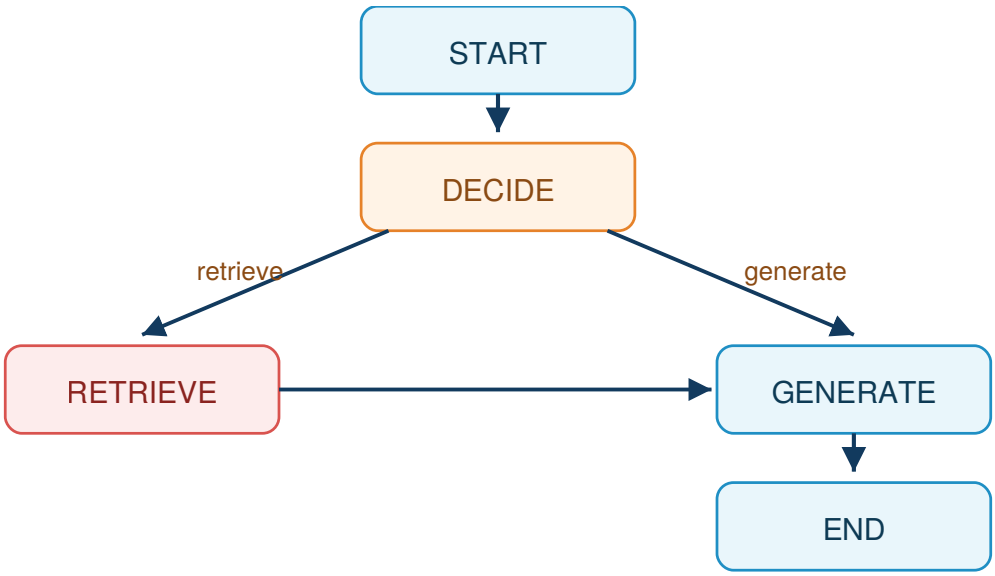

Fig 1. START → DECIDE, then a conditional edge branches to RETRIEVE or straight to GENERATE, and both paths converge before END.

State — the shared memory of the workflow

A workflow needs somewhere to keep track of what's happened so far. In this example, the shared **state** holds: `question`, `documents`, `answer`, and `needs_retrieval`.

The three nodes

Node	What it does
`decide`	Figures out whether retrieval is needed
`retrieve`	Fetches relevant documents (only runs if needed)
`generate`	Produces the final answer, with or without retrieved context

5. The Shared State (Code)

```
from typing import TypedDict, List
from langchain_core.documents import Document

class AgentState(TypedDict):
    question: str
    documents: List[Document]
    answer: str
    needs_retrieval: bool
```

This is just a labeled container: it says exactly what pieces of information flow through the workflow, and what type each one is.

6. The Decide Node (Code)

A simple, rule-based check: does the question contain a word that usually signals "I need real information," like *what*, *how*, or *explain*?

```
def decide_retrieval(state):
    question = state["question"].lower()
    keywords = ["what", "how", "explain",
                "describe", "tell me"]
    needs = any(word in question for word in keywords)
    return {"needs_retrieval": needs}
```

WORTH REMEMBERING

> This keyword check is deliberately simple. The same node could later be swapped out for an LLM call that makes a smarter judgment call — the workflow structure wouldn't need to change, just this one function.

7. The Retrieve Node (Code)

```
def retrieve_documents(state):
    question = state["question"]
    documents = retriever.invoke(question)
    return {"documents": documents}
```

This only runs when `decide` routes the workflow here. It calls the retriever and stores whatever comes back into the shared state.

8. The Generate Node (Code)

```
def generate_answer(state):
    question = state["question"]
    documents = state.get("documents", [])

    context = "\n\n".join(
        doc.page_content for doc in documents
    )

    prompt = f'''
    Based on the following context,
    answer the question.
    Context:
    {context}
    Question:
    {question}
    '''

    response = llm.invoke(prompt)
    return {"answer": response.content}
```

Notice this node works whether or not retrieval happened — if `documents` is empty, `context` just becomes an empty string, and the LLM answers from its own knowledge instead.

9. The Conditional Function (Code)

```
def should_retrieve(state):
    if state["needs_retrieval"]:
        return "retrieve"
    return "generate"
```

> **Q: What does a conditional edge actually do?**
>
> It looks at the current state and decides which node to go to next. Here, `state['needs_retrieval']` being True routes to `retrieve`; False skips straight to `generate`.

10. Wiring the Graph Together (Code)

```
workflow = StateGraph(AgentState)

workflow.add_node("decide", decide_retrieval)
workflow.add_node("retrieve", retrieve_documents)
workflow.add_node("generate", generate_answer)

workflow.add_edge(START, "decide")

workflow.add_conditional_edges(
    "decide",
    should_retrieve,
    {
        "retrieve": "retrieve",
        "generate": "generate"
    }
)

workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)

app = workflow.compile()
```

READ IT LIKE A SENTENCE

"Build a graph with this state shape. Add three nodes. Start at decide. From decide, branch based on should_retrieve. If retrieve happens, always go to generate next. Once generate finishes, we're done."

11. Running the Workflow (Code)

```
initial_state = {
    "question": "What is LangGraph?",
    "documents": [],
    "answer": "",
    "needs_retrieval": False
}

result = app.invoke(initial_state)
print(result["answer"])
```

You always start with a full initial state (even if some fields are just placeholders like empty strings or `False`) — the graph fills in the rest as it runs.

12. Environment Variables & Vector Store

PLACEMENT POINT

The OpenAI API key is loaded from environment variables, not hard-coded. Never commit API keys to GitHub — always use environment variables or a secrets manager.

For retrieval to work at all, documents need to exist somewhere searchable first. The general concept, regardless of exact vector store used:

Fig 2. The exact vector store used isn't clear from the transcript — the concept is what matters.

13. Placement Interview Q&A

Q1. What is Agentic RAG?
RAG combined with an autonomous agent that dynamically decides how retrieval and generation should happen.

Q2. Agentic RAG vs traditional RAG?
Traditional RAG follows a mostly fixed retrieve-and-generate flow. Agentic RAG adds dynamic decision-making on top.

Q3. What decisions can an agent make?
When to retrieve, what to retrieve, where to retrieve, and how many times to retrieve.

Q4. Why LangGraph?
It represents the workflow using nodes, shared state, edges, and conditional edges.

Q5. What is state?
Shared data passed between workflow nodes — here, question, documents, answer, and a retrieval flag.

Q6. What is a node?
A unit of work in the graph, such as deciding, retrieving, or generating.

Q7. What is a conditional edge?
A transition where the next node depends on the current state/decision, rather than always going the same way.

Q8. What does the retriever do?
It finds relevant external documents/context for a given query.

Q9. Can the decision be made without an LLM?
Yes — simple Python keyword rules work fine as a starting point, and can later be replaced by an LLM call.

Q10. What happens after retrieval?
Retrieved documents are stored in state, combined into context, inserted into a prompt, and sent to the LLM.

Q11. What happens if retrieval is false?
The workflow goes directly from the decision node to generation, skipping retrieval entirely.

Q12. What is the role of the agent?
It makes dynamic decisions about actions/retrieval rather than always following one fixed path.

14. Common Interview Confusions

KEEP THESE DISTINCTIONS STRAIGHT

- **Agent ≠ retriever** — the retriever fetches information; the agent decides what action/path to take.
- **State ≠ vector DB** — state carries workflow information; the vector store holds searchable data.
- **LangGraph ≠ LLM** — LangGraph manages the workflow; an LLM may be used inside individual nodes for reasoning/generation.
- **Conditional edge ≠ normal edge** — a conditional edge chooses between multiple possible paths; a normal edge always goes the same way.

15. Final Placement Cheat Sheet

- Agentic RAG = RAG + Autonomous Agent
- Traditional: Query → Retrieve → Context → Prompt → LLM
- Agentic: Query → Decision → Retrieve OR Generate → Generate
- Agent decisions: When? What? Where? How many times?
- LangGraph = nodes + state + edges + conditional edges
- Example nodes: decide → retrieve → generate

- State fields: question, documents, answer, needs_retrieval
- Decide can start as keyword rules, later upgraded to an LLM
- Generate works with or without retrieved context
- Keep API keys in environment variables, never in source code

16. Final 30-Second Interview Answer

> *"Agentic RAG combines Retrieval-Augmented Generation with an autonomous agent. Traditional RAG generally follows a fixed retrieve-and-generate pipeline, while Agentic RAG allows an agent to dynamically decide whether retrieval is needed, what or where to retrieve from, and potentially how many retrieval steps are required. In the demonstrated implementation, LangGraph manages the workflow using shared state, nodes and conditional edges such as decide, retrieve and generate."*

17. One-Minute Code Explanation

> *"First, I define the workflow state containing the question, documents, answer and retrieval flag. Then I create three nodes: decide, retrieve and generate. The decide node determines whether retrieval is required. A conditional edge checks the flag. If true, the workflow retrieves documents and then generates an answer; otherwise it goes directly to generation. The generation node combines retrieved documents into context and invokes the LLM. Finally, the graph is compiled and invoked with an initial state."*